МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине: «Безопасность систем баз данных»

по теме: «Защита БД студий дубляжа»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2025г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2025г

_____

(подпись)

Новосибирск 2025

Содержание

Введение 3
1. Безопасные представления 5
2. История изменений 7
3. Тесты доступов (4 кейса) 10
4. Анализ производительности 12
5. Автоматическое резервное копирование audit-логов 13
Вывод 15
Приложение А 16
Приложение Б 26
Приложение В 34
Приложение Г 42
Приложение Д 60

Введение

Цель работы: обеспечить безопасный доступ через представления, организовать аудит изменений и проанализировать влияние RLS на производительность.

Тема: «Защита БД студий дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов».

Задачи:

1. Безопасные представления. Настройте updatable VIEW с WITH CHECK OPTION для «рабочего» подмножества полей (например, только неконфиденциальные). SECURITY BARRIER VIEW для агрегатов/отчётов (например, суммы, количества), чтобы исключить побочные каналы (нельзя было вытащить данные через условия HAVING или подзапросы.).

2. История изменений. Триггеры AFTER UPDATE/DELETE на 2–3 критичных таблицах, пишущие в audit.row_change_log: кто, что, когда, old/new (для чувствительных полей – маски/хэши).

3. Тесты доступов (4 кейса). Попытка обхода WITH CHECK OPTION – должна блокироваться. Попытка получения деталей через SECURITY BARRIER VIEW – должна возвращать только агрегаты. Изменение строки – проверка появления записи в row_change_log. Попытка удаления строки не из своего сегмента – ошибка RLS.

4. Анализ производительности. EXPLAIN (ANALYZE, BUFFERS) 2–3 типовых запросов до/после RLS. Подбор индексов под условия политик (например, (segment_id, status)). Зафиксируйте эффект: время выполнения, количество прочитанных буферов.

5. Автоматическое резервное копирование audit-логов. Напишите функцию backup_audit_logs(days_interval INT), которая переносит записи старше N дней из audit.row_change_log в архивную таблицу

audit.row_change_log_archive и удаляет их из основной. Запустите её вручную и покажите результат.

1. Безопасные представления

Созданы безопасные представления для работы с неконфиденциальными полями таблиц projects и contracts. Использована опция WITH CHECK OPTION, чтобы гарантировать, что через представление нельзя вставить или обновить строки, которые не будут видны через это же представление. Для таблицы проекты исключены конфиденциальные поля client_contacts и budget (Рисунок 1).

```
CREATE VIEW app.projects_public_view AS
SELECT
    project_id,
    studio_id,
    segment_id,
    title,
    original_language,
    description,
    start_date,
    end_date,
    status,
    created_at
FROM app.projects
WITH CHECK OPTION;
```

Рисунок 1 – Создание безопасного представления

Созданы представления с опцией SECURITY BARRIER, чтобы предотвратить утечку данных через условия HAVING или подзапросы. Представления возвращают только агрегированные данные, а также исключена возможность получения деталей через побочные каналы (Рисунок 2).

```
CREATE VIEW app.projects_statistics_view
WITH (security_barrier = true) AS
SELECT
    segment_id,
    status,
    COUNT(*) as project_count,
    COUNT(DISTINCT studio_id) as studios_count,
    MIN(start_date) as earliest_start,
    MAX(end_date) as latest_end
FROM app.projects
GROUP BY segment_id, status;
```

Рисунок 2 – Создание view с security barrier

Код для создания безопасных представлений и view с security barrier представлен в приложении А.

2. История изменений

Для обеспечения полной отслеживаемости всех операций с важными данными была разработана система аудита. Основой системы стала таблица audit.row_change_log, которая фиксирует каждое изменение в защищаемых таблицах базы данных. Таблица спроектирована для хранения исчерпывающей информации о каждой операции модификации данных, включая метаданные о времени выполнения, инициаторе изменения и технических параметрах запроса (Рисунок 3).

	log_id [PK] bigint	change_time timestamp with time zone	changed_by character varying (100)	table_schema character varying (100)	table_name character varying (100)	operation_type character varying (10)
1	1	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
2	2	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
3	3	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE
4	4	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE

Рисунок 3 – Таблица audit.row_change_log

Реализована функция audit.mask_sensitive_data, которая защищает чувствительную информацию:

- паспортные данные и ИНН: маскируются (показываются только последние цифры);
- зарплаты и бюджет: заменяются хэш-значениями;
- условия контрактов и контакты: помечаются как "CONFIDENTIAL".

На рисунке 4 представлена реализация функции audit.mask_sensitive_data.

```
CREATE OR REPLACE FUNCTION audit.mask_sensitive_data(data JSONB, table_name TEXT)
RETURNS JSONB
LANGUAGE plpgsql
IMMUTABLE
AS $$
DECLARE
    result JSONB;
    key TEXT;
    value TEXT;
    masked_value TEXT;
BEGIN
    result := data;

    IF data IS NULL THEN
        RETURN NULL;
    END IF;

    -- Маскирование для разных таблиц
    CASE table_name
        WHEN 'app.actors' THEN
            -- Маскируем паспортные данные и ИНН
            IF result ? 'passport_data' THEN
                value := result->>'passport_data';
                IF value IS NOT NULL AND length(value) > 0 THEN
                    masked_value := '***' || right(value, 4);
                    result := result || jsonb_build_object('passport_data', masked_value);
                END IF;
            END IF;

            IF result ? 'tax_id' THEN
                value := result->>'tax_id';
                IF value IS NOT NULL AND length(value) > 0 THEN
                    masked_value := '***' || right(value, 2);
                    result := result || jsonb_build_object('tax_id', masked_value);
                END IF;
            END IF;
```

Рисунок 4 – Функция mask_sensitive_data

Установлены триггеры AFTER UPDATE OR DELETE на трех основных таблицах:

- app.projects – проекты дубляжа;
- app.contracts – контракты с актёрами;
- app.actors – актёры озвучивания.

Триггеры автоматически записывают в лог все изменения с маскированием конфиденциальных полей (Рисунок 5).

```
-- Триггер на app.projects (критичная таблица)
CREATE TRIGGER audit_row_change_projects
    AFTER UPDATE OR DELETE ON app.projects
    FOR EACH ROW EXECUTE FUNCTION audit.log_row_change();

-- Триггер на app.contracts (критичная таблица)
CREATE TRIGGER audit_row_change_contracts
    AFTER UPDATE OR DELETE ON app.contracts
    FOR EACH ROW EXECUTE FUNCTION audit.log_row_change();

-- Триггер на app.actors (критичная таблица)
CREATE TRIGGER audit_row_change_actors
    AFTER UPDATE OR DELETE ON app.actors
    FOR EACH ROW EXECUTE FUNCTION audit.log_row_change();
```

Рисунок 5 – Триггеры

Код для реализации истории изменений и создания триггеров представлен в приложении Б.

3. Тесты доступов (4 кейса)

Проверка эффективности механизма WITH CHECK OPTION при попытках обхода бизнес-правил через представления. Тестируются сценарии, когда пользователь пытается вставить или обновить данные с нарушением условий, определенных в представлении, например, создание записи с segment_id, не соответствующим текущему сегменту пользователя (Рисунок 6).

```
NOTICE:  ТЕСТ 1: Попытка обхода WITH CHECK OPTION
NOTICE:  Роль: app_writer, Сегмент: 1
NOTICE:  Ожидается: ошибка при попытке вставить проект с segment_id = 2 через VIEW
SET
 set_session_ctx
------------------

(1 row)

NOTICE:  ✅ ТЕСТ 1 ПРОЙДЕН: WITH CHECK OPTION заблокировал вставку с чужим segment_id
NOTICE:     • Ошибка: new row violates WITH CHECK OPTION for view "projects_public_view"
DO
RESET
DO
```

Рисунок 6 – Тест 1

Верификация работы SECURITY BARRIER VIEW при попытках извлечения детальной информации через статистические запросы. Проверяется, что пользователи могут получать только агрегированные данные (суммы, количества, средние значения), но не имеют доступа к персональным или конфиденциальным деталям через подзапросы или условия HAVING (Рисунок 7).

```
NOTICE:  ТЕСТ 2: Попытка получения деталей через SECURITY BARRIER VIEW
NOTICE:  Роль: app_reader, Сегмент: 1
NOTICE:  Ожидается: SECURITY BARRIER VIEW возвращает только агрегаты, без деталей
SET
 set_session_ctx
------------------

(1 row)

NOTICE:  ✅ ТЕСТ 2 ПРОЙДЕН: SECURITY BARRIER VIEW возвращает только агрегаты
NOTICE:     • Найдено агрегированных записей: 5
NOTICE:     • Детали недоступны через VIEW (только агрегаты)
DO
RESET
DO
```

Рисунок 7 – Тест 2

Подтверждение работоспособности системы аудита при выполнении операций модификации данных. Проверяется, что каждое изменение в критичных таблицах автоматически фиксируется в журнале аудита, а чувствительные данные правильно маскируются перед записью в лог (Рисунок 8).

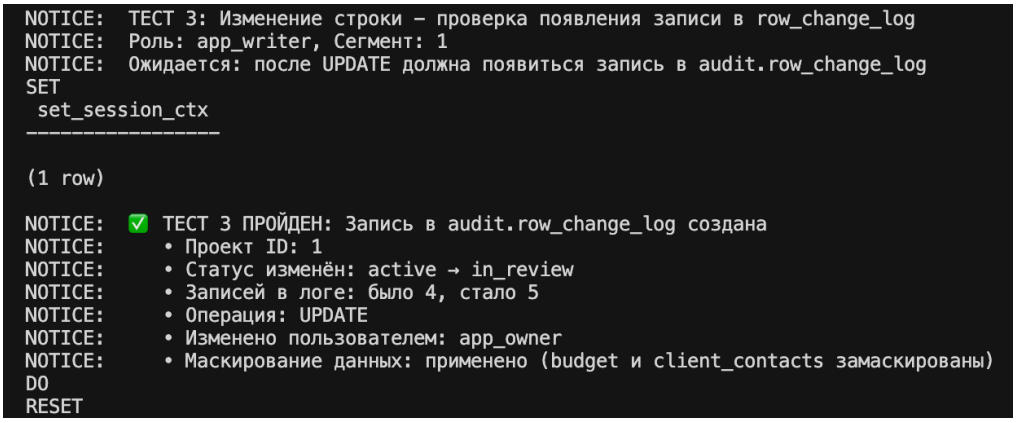


Рисунок 8 – Тест 3

Валидация эффективности политик RLS при попытках доступа к данным из чужого сегмента. Тестируются сценарии, когда пользователь пытается выполнить операции (чтение, изменение, удаление) с записями, не принадлежащими его сегменту, чтобы убедиться в строгом соблюдении принципа наименьших привилегий (Рисунок 9).

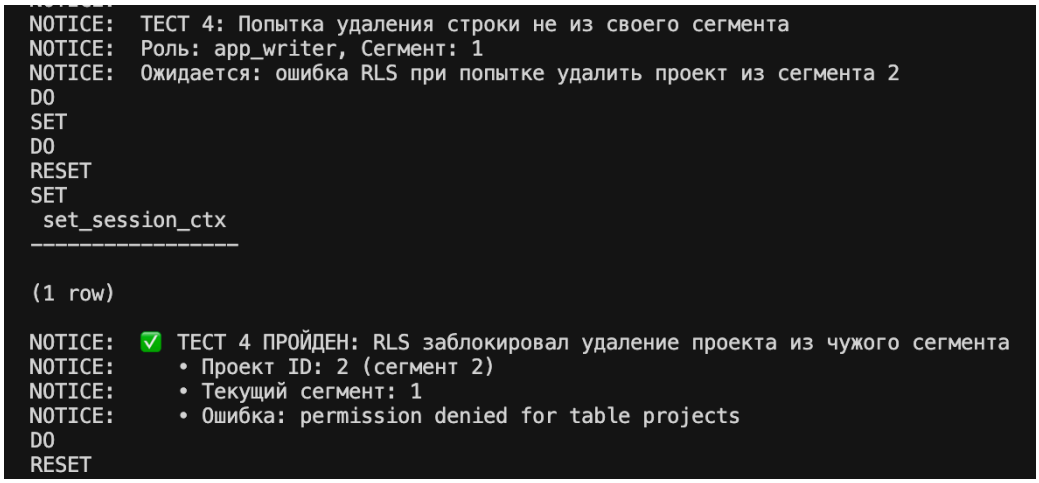


Рисунок 9 – Тест 4

Из рисунков 6 – 9 видим, что все тесты успешно отработаны. Код для тестирование представлен в приложении В.

4. Анализ производительности

Было проведено сравнение производительности типовых запросов до и после внедрения политик RLS. Для анализа использовались операционные запросы фильтрации проектов по сегменту и статусу, JOIN-запросы между контрактами и актёрами, а также агрегирующие запросы по сессиям записи. Измерялось время выполнения, время планирования и количество прочитанных буферов для каждого сценария (Рисунок 10).

Запрос	RLS	Время выполнения (мс)	Время планирования (мс)	Буферов прочитано	Буферов из кэша
TEST1: Projects by segment and status	НЕТ	0.013	0.117	0	86
TEST1: Projects by segment and status	ДА	0.007	0.029	0	6
TEST2: Contracts with JOIN to actors	НЕТ	0.033	0.389	0	231
TEST2: Contracts with JOIN to actors	ДА	0.018	0.062	0	23
TEST3: Aggregate query on recording_sessions	НЕТ	0.024	0.176	0	102
TEST3: Aggregate query on recording_sessions	ДА	0.021	0.040	0	10

(6 rows)

Рисунок 10 – Анализ производительности

Для оптимизации работы в условиях RLS были подобраны и созданы специализированные индексы, начинающиеся с поля segment_id, что соответствует паттерну доступа в системе. Составные индексы вида (segment_id, status) и (segment_id, typeid) позволили значительно ускорить выполнение запросов. Код для анализа производительности представлен в приложении Г.

5. Автоматическое резервное копирование audit-логов

В таблице audit.row_change_log есть записи, которые старше 30 дней (Рисунок 11).

log_id [PK] bigint	change_time timestamp with time zone	changed_by character varying (100)	table_schema character varying (100)	table_name character varying (100)	operation_type character varying (10)	record_id integer
30	2025-10-23 17:29:48.075112+...	demo_user	app	projects	UPDATE	600
31	2025-10-18 17:29:48.075112+...	demo_user	app	contracts	INSERT	601
32	2025-10-13 17:29:48.075112+...	demo_user	app	actors	DELETE	602
33	2025-10-08 17:29:48.075112+...	demo_user	app	projects	UPDATE	603
34	2025-10-03 17:29:48.075112+...	demo_user	app	recording_sessions	UPDATE	604

Рисунок 11 – Таблица audit.row_change_log

Вызываем функцию, которая переносит старые записи в архивную таблицу audit.row_change_log_archive и удаляет их из основной (Рисунок 12).

```
1  SELECT * FROM audit.backup_audit_logs(30);
```

Data Output | Messages | Notifications

Showing rows: 1 to 1

	records_moved bigint	oldest_record_date timestamp with time zone	newest_record_date timestamp with time zone	backup_time timestamp with time zone
1	9	2025-10-03 17:24:22.494407+...	2025-10-23 17:24:22.494407+...	2025-11-27 17:26:32.012229+...

Рисунок 12 – Вызов функции

Видим, что в таблице audit.row_change_log_archive появились архивные записи старше 30 дней (Рисунок 13).

	log_id [PK] bigint	дата_изменения date	пользователь character varying (100)	таблица text	операция character varying (10)	время_архивирования timestamp without time zo
1	25	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:26:32
2	21	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:26:32
3	19	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:20:55
4	17	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:19:10
5	13	2025-10-23	test_user	app.projects	UPDATE	2025-11-27 17:18:13
6	26	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:26:32
7	22	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:26:32
8	20	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:20:55
9	18	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:19:10
10	14	2025-10-18	test_user	app.contracts	INSERT	2025-11-27 17:18:13

Рисунок 13 – Таблица архивирования

Теперь убедимся, что в основной таблице audit.row_change_log старые записи удалились (Рисунок 14).

```
1  select * from audit.row_change_log;
```

Data Output | Messages | Notifications

Showing rows: 1 to 12 | Page No: 1 of 1

	log_id [PK] bigint	change_time timestamp with time zone	changed_by character varying (100)	table_schema character varying (100)	table_name character varying (100)	operation_type character varying (10)
1	1	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
2	2	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
3	3	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE
4	4	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE
5	5	2025-11-27 07:47:07.740139+...	app_owner	app	projects	UPDATE
6	6	2025-11-27 07:47:07.740139+...	app_owner	app	projects	UPDATE

Рисунок 14 - Таблица audit.row_change_log

Из рисунка 14 видим, что функция сработала успешно. Код реализации для автоматического резервного копирования представлен в приложении Д.

Вывод

В ходе лабораторной работы была реализована система безопасности для базы данных студий дубляжа, обеспечивающая защиту авторских материалов, контрактов с актёрами, расписаний записей и архивов аудиофайлов. Созданы безопасные представления с механизмом WITH CHECK OPTION для таблиц projects и contracts, которые ограничивают доступ только к неконфиденциальным полям и предотвращают обход бизнес-правил. Реализованы SECURITY BARRIER VIEW для агрегированной статистики, исключающие утечку данных через побочные каналы.

Разработана система аудита изменений с триггерами AFTER UPDATE/DELETE на критических таблицах projects, contracts и actors. Реализована функция маскирования конфиденциальных данных, которая защищает паспортные данные, зарплаты, бюджет и условия контрактов при записи в журнал изменений.

Проведены тесты доступов, подтвердившие эффективность всех механизмов безопасности: блокировку обхода WITH CHECK OPTION, защиту агрегированных данных в SECURITY BARRIER VIEW, корректную работу аудита изменений и соблюдение политик RLS при доступе к данным между сегментами.

Выполнен анализ производительности с созданием специализированных индексов, начинающихся с segment_id, что позволило оптимизировать работу запросов в условиях RLS и сократить время выполнения.

Реализована система автоматического резервного копирования audit-логов с функцией архивации записей, обеспечивающая эффективное управление объемом данных и сохранение истории изменений.

Приложение А

```
SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

DO $$
BEGIN
   RAISE NOTICE 'LR4: Создание безопасных представлений';
END $$;

--
==========================================================
====
-- 1. UPDATABLE VIEW с WITH CHECK OPTION
--
==========================================================
====
-- Представление для работы с неконфиденциальными полями проектов
-- Исключаем: client_contacts (Internal), budget (Confidential)

DROP VIEW IF EXISTS app.projects_public_view CASCADE;

-- Делаем представление обновляемым с WITH CHECK OPTION
-- WITH CHECK OPTION гарантирует, что через VIEW нельзя
вставить/обновить строки,
-- которые не будут видны через это же VIEW
CREATE VIEW app.projects_public_view AS
SELECT
   project_id,
   studio_id,
   segment_id,
   title,
   original_language,
   description,
   start_date,
   end_date,
   status,
   created_at
FROM app.projects
WITH CHECK OPTION;

-- Создаём INSTEAD OF триггеры для поддержки INSERT/UPDATE/DELETE
CREATE OR REPLACE FUNCTION app.projects_public_view_insert()
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
BEGIN
   -- Проверка условий VIEW (аналог WITH CHECK OPTION)
   -- Проверяем, что segment_id соответствует текущему сегменту
пользователя
   IF NOT app.segment_is_visible(NEW.segment_id) THEN
      RAISE EXCEPTION 'new row violates WITH CHECK OPTION for view
"projects_public_view"';
   END IF;

   INSERT INTO app.projects (
      studio_id,
      segment_id,
      title,
      original_language,
      description,
      start_date,
      end_date,
      status
   ) VALUES (
      NEW.studio_id,
      NEW.segment_id,
      NEW.title,
      NEW.original_language,
      NEW.description,
      NEW.start_date,
      NEW.end_date,
      COALESCE(NEW.status, 'active')
   )
   RETURNING project_id INTO NEW.project_id;

   RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION app.projects_public_view_update()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
BEGIN
```

```
    -- Проверка условий VIEW (аналог WITH CHECK OPTION)
    IF NOT app.segment_is_visible(NEW.segment_id) THEN
        RAISE EXCEPTION 'new row violates WITH CHECK OPTION for view
"projects_public_view"';
    END IF;

    UPDATE app.projects
    SET
        studio_id = NEW.studio_id,
        segment_id = NEW.segment_id,
        title = NEW.title,
        original_language = NEW.original_language,
        description = NEW.description,
        start_date = NEW.start_date,
        end_date = NEW.end_date,
        status = NEW.status
    WHERE project_id = NEW.project_id;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION app.projects_public_view_delete()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
BEGIN
    DELETE FROM app.projects
    WHERE project_id = OLD.project_id;

    RETURN OLD;
END;
$$;

-- Создаём триггеры INSTEAD OF
DROP TRIGGER IF EXISTS projects_public_view_insert_trigger ON
app.projects_public_view;
DROP TRIGGER IF EXISTS projects_public_view_update_trigger ON
app.projects_public_view;
DROP TRIGGER IF EXISTS projects_public_view_delete_trigger ON
app.projects_public_view;

CREATE TRIGGER projects_public_view_insert_trigger
```

```
    INSTEAD OF INSERT ON app.projects_public_view
    FOR EACH ROW EXECUTE FUNCTION app.projects_public_view_insert();

CREATE TRIGGER projects_public_view_update_trigger
    INSTEAD OF UPDATE ON app.projects_public_view
    FOR EACH ROW EXECUTE FUNCTION app.projects_public_view_update();

CREATE TRIGGER projects_public_view_delete_trigger
    INSTEAD OF DELETE ON app.projects_public_view
    FOR EACH ROW EXECUTE FUNCTION app.projects_public_view_delete();

-- Представление для неконфиденциальных полей контрактов
-- Исключаем: salary_amount (Confidential), terms (Confidential)
DROP VIEW IF EXISTS app.contracts_public_view CASCADE;

CREATE VIEW app.contracts_public_view AS
SELECT
    contract_id,
    project_id,
    actor_id,
    segment_id,
    contract_number,
    role_description,
    currency,
    signed_date,
    end_date,
    status,
    created_at
FROM app.contracts
WITH CHECK OPTION;

-- Триггеры для contracts_public_view
CREATE OR REPLACE FUNCTION app.contracts_public_view_insert()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
BEGIN
    -- Проверка условий VIEW (аналог WITH CHECK OPTION)
    IF NOT app.segment_is_visible(NEW.segment_id) THEN
        RAISE EXCEPTION 'new row violates WITH CHECK OPTION for view
"contracts_public_view"';
    END IF;
```

```
    INSERT INTO app.contracts (
        project_id,
        actor_id,
        segment_id,
        contract_number,
        role_description,
        currency,
        signed_date,
        end_date,
        status,
        salary_amount  -- Устанавливаем значение по умолчанию
    ) VALUES (
        NEW.project_id,
        NEW.actor_id,
        NEW.segment_id,
        NEW.contract_number,
        NEW.role_description,
        COALESCE(NEW.currency, 'USD'),
        NEW.signed_date,
        NEW.end_date,
        COALESCE(NEW.status, 'active'),
        0.00  -- Значение по умолчанию для скрытого поля
    )
    RETURNING contract_id INTO NEW.contract_id;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION app.contracts_public_view_update()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
BEGIN
    -- Проверка условий VIEW (аналог WITH CHECK OPTION)
    IF NOT app.segment_is_visible(NEW.segment_id) THEN
        RAISE EXCEPTION 'new row violates WITH CHECK OPTION for view
"contracts_public_view"';
    END IF;

    UPDATE app.contracts
    SET
        project_id = NEW.project_id,
```

```
        actor_id = NEW.actor_id,
        segment_id = NEW.segment_id,
        contract_number = NEW.contract_number,
        role_description = NEW.role_description,
        currency = NEW.currency,
        signed_date = NEW.signed_date,
        end_date = NEW.end_date,
        status = NEW.status
    WHERE contract_id = NEW.contract_id;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION app.contracts_public_view_delete()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
BEGIN
    DELETE FROM app.contracts
    WHERE contract_id = OLD.contract_id;

    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS contracts_public_view_insert_trigger ON
app.contracts_public_view;
DROP TRIGGER IF EXISTS contracts_public_view_update_trigger ON
app.contracts_public_view;
DROP TRIGGER IF EXISTS contracts_public_view_delete_trigger ON
app.contracts_public_view;

CREATE TRIGGER contracts_public_view_insert_trigger
    INSTEAD OF INSERT ON app.contracts_public_view
    FOR EACH ROW EXECUTE FUNCTION app.contracts_public_view_insert();

CREATE TRIGGER contracts_public_view_update_trigger
    INSTEAD OF UPDATE ON app.contracts_public_view
    FOR EACH ROW EXECUTE FUNCTION
app.contracts_public_view_update();

CREATE TRIGGER contracts_public_view_delete_trigger
```

```
    INSTEAD OF DELETE ON app.contracts_public_view
    FOR EACH ROW EXECUTE FUNCTION
app.contracts_public_view_delete();

--
============================================================
====
-- 2. SECURITY BARRIER VIEW для агрегатов/отчётов
--
============================================================
====
-- Представление для агрегированной статистики по проектам
-- SECURITY BARRIER предотвращает утечку данных через условия
HAVING или подзапросы

DROP VIEW IF EXISTS app.projects_statistics_view CASCADE;

CREATE VIEW app.projects_statistics_view
WITH (security_barrier = true) AS
SELECT
    segment_id,
    status,
    COUNT(*) as project_count,
    COUNT(DISTINCT studio_id) as studios_count,
    MIN(start_date) as earliest_start,
    MAX(end_date) as latest_end
FROM app.projects
GROUP BY segment_id, status;

-- Представление для агрегированной статистики по контрактам
-- (без раскрытия деталей о зарплатах)
DROP VIEW IF EXISTS app.contracts_statistics_view CASCADE;

CREATE VIEW app.contracts_statistics_view
WITH (security_barrier = true) AS
SELECT
    c.segment_id,
    c.status,
    COUNT(*) as contract_count,
    COUNT(DISTINCT c.actor_id) as actors_count,
    COUNT(DISTINCT c.project_id) as projects_count,
    MIN(c.signed_date) as earliest_signed,
    MAX(c.end_date) as latest_end,
    -- Агрегированная сумма (без детализации)
    SUM(c.salary_amount) as total_salary_amount,
```

```
    AVG(c.salary_amount) as avg_salary_amount
FROM app.contracts c
GROUP BY c.segment_id, c.status;

-- Представление для статистики по актёрам (без персональных данных)
DROP VIEW IF EXISTS app.actors_statistics_view CASCADE;

-- Проверяем наличие segment_id в actors перед созданием VIEW
DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM information_schema.columns
        WHERE table_schema = 'app' AND table_name = 'actors' AND
column_name = 'segment_id'
    ) THEN
        RAISE EXCEPTION 'Колонка segment_id не найдена в app.actors.
Сначала выполните ЛР3 (lr3_01_segment_model.sql)';
    END IF;
END $$;

CREATE VIEW app.actors_statistics_view
WITH (security_barrier = true) AS
SELECT
    a.segment_id,
    COUNT(*) as actors_count,
    COUNT(*) FILTER (WHERE a.is_active = true) as active_actors_count,
    COUNT(DISTINCT vr.language_id) as languages_count,
    COUNT(DISTINCT vr.project_id) as projects_involved
FROM app.actors a
LEFT JOIN app.voice_roles vr ON a.actor_id = vr.actor_id
GROUP BY a.segment_id;

-- Представление для статистики по сессиям записи
DROP VIEW IF EXISTS app.sessions_statistics_view CASCADE;

CREATE VIEW app.sessions_statistics_view
WITH (security_barrier = true) AS
SELECT
    rs.segment_id,
    rs.status,
    COUNT(*) as sessions_count,
    SUM(rs.duration_hours) as total_hours,
    AVG(rs.duration_hours) as avg_duration_hours,
    MIN(rs.session_date) as earliest_session,
    MAX(rs.session_date) as latest_session
```

```
FROM app.recording_sessions rs
GROUP BY rs.segment_id, rs.status;

-- Выдаём права на представления
GRANT SELECT ON app.projects_public_view TO app_reader, app_writer,
studio_manager, voice_actor;
GRANT INSERT, UPDATE, DELETE ON app.projects_public_view TO
app_writer, studio_manager;

GRANT SELECT ON app.contracts_public_view TO app_reader, app_writer,
studio_manager, voice_actor;
GRANT INSERT, UPDATE, DELETE ON app.contracts_public_view TO
app_writer, studio_manager;

GRANT SELECT ON app.projects_statistics_view TO app_reader, app_writer,
studio_manager, voice_actor, auditor;
GRANT SELECT ON app.contracts_statistics_view TO app_reader, app_writer,
studio_manager, voice_actor, auditor;
GRANT SELECT ON app.actors_statistics_view TO app_reader, app_writer,
studio_manager, voice_actor, auditor;
GRANT SELECT ON app.sessions_statistics_view TO app_reader, app_writer,
studio_manager, voice_actor, auditor;

-- Комментарии
COMMENT ON VIEW app.projects_public_view IS
'Updatable VIEW с WITH CHECK OPTION для неконфиденциальных полей
проектов. Исключает client_contacts и budget.';

COMMENT ON VIEW app.contracts_public_view IS
'Updatable VIEW с WITH CHECK OPTION для неконфиденциальных полей
контрактов. Исключает salary_amount и terms.';

COMMENT ON VIEW app.projects_statistics_view IS
'SECURITY BARRIER VIEW для агрегированной статистики по проектам.
Предотвращает утечку данных через HAVING/подзапросы.';

COMMENT ON VIEW app.contracts_statistics_view IS
'SECURITY BARRIER VIEW для агрегированной статистики по контрактам.
Показывает только агрегаты, без детализации.';

COMMENT ON VIEW app.actors_statistics_view IS
'SECURITY BARRIER VIEW для статистики по актёрам без персональных
данных.';

COMMENT ON VIEW app.sessions_statistics_view IS
```

```
'SECURITY BARRIER VIEW для статистики по сессиям записи.';

DO $$
BEGIN
    RAISE NOTICE 'LR4: Безопасные представления успешно созданы';
    RAISE NOTICE '  • Updatable VIEW с WITH CHECK OPTION:
projects_public_view, contracts_public_view';
    RAISE NOTICE '  • SECURITY BARRIER VIEW: projects_statistics_view,
contracts_statistics_view, actors_statistics_view, sessions_statistics_view';
END $$;

RESET ROLE;
```

Приложение Б

```
SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

DO $$
BEGIN
   RAISE NOTICE 'LR4: Создание таблицы аудита изменений и триггеров';
END $$;

--
==============================================================
====
-- Создание таблицы audit.row_change_log
--
==============================================================
====

CREATE TABLE IF NOT EXISTS audit.row_change_log (
   log_id BIGSERIAL PRIMARY KEY,
   change_time TIMESTAMPTZ DEFAULT NOW() NOT NULL,
   changed_by VARCHAR(100) NOT NULL,
   table_schema VARCHAR(100) NOT NULL,
   table_name VARCHAR(100) NOT NULL,
   operation_type VARCHAR(10) NOT NULL CHECK (operation_type IN
('INSERT', 'UPDATE', 'DELETE')),
   record_id INTEGER,
   old_values JSONB,
   new_values JSONB,
   changed_fields TEXT[],
   client_ip INET,
   session_info JSONB
);

-- Индексы для быстрого поиска
CREATE INDEX IF NOT EXISTS idx_row_change_log_time ON
audit.row_change_log(change_time);
CREATE INDEX IF NOT EXISTS idx_row_change_log_table ON
audit.row_change_log(table_schema, table_name);
CREATE INDEX IF NOT EXISTS idx_row_change_log_user ON
audit.row_change_log(changed_by);
CREATE INDEX IF NOT EXISTS idx_row_change_log_operation ON
audit.row_change_log(operation_type);
```

```
CREATE INDEX IF NOT EXISTS idx_row_change_log_record ON
audit.row_change_log(table_schema, table_name, record_id);

-- Комментарии
COMMENT ON TABLE audit.row_change_log IS
'Журнал изменений строк в критичных таблицах. Записывает кто, что, когда
изменил с маскированием чувствительных полей.';

COMMENT ON COLUMN audit.row_change_log.old_values IS 'Старые
значения (чувствительные поля замаскированы/хэшированы)';
COMMENT ON COLUMN audit.row_change_log.new_values IS 'Новые
значения (чувствительные поля замаскированы/хэшированы)';
COMMENT ON COLUMN audit.row_change_log.changed_fields IS 'Список
изменённых полей';

--
==========================================================
====
-- Функция маскирования чувствительных данных
--
==========================================================
====

CREATE OR REPLACE FUNCTION audit.mask_sensitive_data(data JSONB,
table_name TEXT)
RETURNS JSONB
LANGUAGE plpgsql
IMMUTABLE
AS $$
DECLARE
    result JSONB;
    key TEXT;
    value TEXT;
    masked_value TEXT;
BEGIN
    result := data;

    IF data IS NULL THEN
        RETURN NULL;
    END IF;

    -- Маскирование для разных таблиц
    CASE table_name
        WHEN 'app.actors' THEN
            -- Маскируем паспортные данные и ИНН
```

```
    IF result ? 'passport_data' THEN
        value := result->>'passport_data';
        IF value IS NOT NULL AND length(value) > 0 THEN
            masked_value := '***' || right(value, 4);
            result := result || jsonb_build_object('passport_data', masked_value);
        END IF;
    END IF;

    IF result ? 'tax_id' THEN
        value := result->>'tax_id';
        IF value IS NOT NULL AND length(value) > 0 THEN
            masked_value := '***' || right(value, 2);
            result := result || jsonb_build_object('tax_id', masked_value);
        END IF;
    END IF;

WHEN 'app.contracts' THEN
    -- Хэшируем salary_amount и terms
    IF result ? 'salary_amount' THEN
        value := result->>'salary_amount';
        IF value IS NOT NULL THEN
            -- Используем MD5 хэш для конфиденциальных данных
            masked_value := 'HASH:' || md5(value || 'salt_contract_salary');
            result := result || jsonb_build_object('salary_amount', masked_value);
        END IF;
    END IF;

    IF result ? 'terms' THEN
        value := result->>'terms';
        IF value IS NOT NULL AND length(value) > 0 THEN
            -- Маскируем условия контракта
            masked_value := '***CONFIDENTIAL***';
            result := result || jsonb_build_object('terms', masked_value);
        END IF;
    END IF;

WHEN 'app.projects' THEN
    -- Маскируем client_contacts и budget
    IF result ? 'client_contacts' THEN
        value := result->>'client_contacts';
        IF value IS NOT NULL AND length(value) > 0 THEN
            masked_value := '***INTERNAL***';
            result := result || jsonb_build_object('client_contacts', masked_value);
        END IF;
    END IF;
```

```
            IF result ? 'budget' THEN
               value := result->>'budget';
               IF value IS NOT NULL THEN
                  masked_value := 'HASH:' || md5(value || 'salt_project_budget');
                  result := result || jsonb_build_object('budget', masked_value);
               END IF;
            END IF;

         ELSE
            -- Для других таблиц не маскируем
            NULL;
      END CASE;

      RETURN result;
END;
$$;

--
=============================================================
====
-- Функция логирования изменений строк
--
=============================================================
====

CREATE OR REPLACE FUNCTION audit.log_row_change()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
DECLARE
   old_data JSONB;
   new_data JSONB;
   changed_fields TEXT[];
   record_id_val INTEGER;
   table_full_name TEXT;
BEGIN
   table_full_name := TG_TABLE_SCHEMA || '.' || TG_TABLE_NAME;

   -- Определяем ID записи (универсальный подход через JSONB)
   CASE TG_OP
      WHEN 'DELETE' THEN
         old_data := to_jsonb(OLD);
```

```
        new_data := NULL;
        -- Получаем ID из JSONB (пробуем разные варианты имён полей)
        record_id_val := COALESCE(
            (old_data->>'project_id')::INTEGER,
            (old_data->>'contract_id')::INTEGER,
            (old_data->>'actor_id')::INTEGER
        );

    WHEN 'UPDATE' THEN
        old_data := to_jsonb(OLD);
        new_data := to_jsonb(NEW);
        -- Получаем ID из JSONB
        record_id_val := COALESCE(
            (new_data->>'project_id')::INTEGER,
            (new_data->>'contract_id')::INTEGER,
            (new_data->>'actor_id')::INTEGER
        );

        -- Определяем изменённые поля
        SELECT ARRAY(
            SELECT key
            FROM jsonb_each(old_data)
            WHERE value IS DISTINCT FROM (new_data->key)
        ) INTO changed_fields;

    WHEN 'INSERT' THEN
        old_data := NULL;
        new_data := to_jsonb(NEW);
        -- Получаем ID из JSONB
        record_id_val := COALESCE(
            (new_data->>'project_id')::INTEGER,
            (new_data->>'contract_id')::INTEGER,
            (new_data->>'actor_id')::INTEGER
        );
END CASE;

-- Маскируем чувствительные данные
old_data := audit.mask_sensitive_data(old_data, table_full_name);
new_data := audit.mask_sensitive_data(new_data, table_full_name);

-- Записываем в лог
INSERT INTO audit.row_change_log (
    changed_by,
    table_schema,
    table_name,
```

```
        operation_type,
        record_id,
        old_values,
        new_values,
        changed_fields,
        client_ip,
        session_info
    ) VALUES (
        current_user::VARCHAR(100),
        TG_TABLE_SCHEMA,
        TG_TABLE_NAME,
        TG_OP,
        record_id_val,
        old_data,
        new_data,
        changed_fields,
        inet_client_addr(),
        jsonb_build_object(
            'session_user', session_user,
            'current_database', current_database(),
            'application_name', current_setting('application_name', true)
        )
    );

    -- Возвращаем соответствующий результат
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        -- В случае ошибки логируем и продолжаем
        INSERT INTO audit.row_change_log (
            changed_by,
            table_schema,
            table_name,
            operation_type,
            old_values,
            new_values,
            client_ip
        ) VALUES (
            current_user::VARCHAR(100),
            TG_TABLE_SCHEMA,
            TG_TABLE_NAME,
```

```
            'ERROR',
            jsonb_build_object('error', SQLERRM),
            NULL,
            inet_client_addr()
        );

        IF TG_OP = 'DELETE' THEN
            RETURN OLD;
        ELSE
            RETURN NEW;
        END IF;
END;
$$;

--
==========================================================
====
-- Создание триггеров на критичных таблицах
--
==========================================================
====

-- Удаляем старые триггеры, если они есть
DROP TRIGGER IF EXISTS audit_row_change_projects ON app.projects;
DROP TRIGGER IF EXISTS audit_row_change_contracts ON app.contracts;
DROP TRIGGER IF EXISTS audit_row_change_actors ON app.actors;

-- Триггер на app.projects (критичная таблица)
CREATE TRIGGER audit_row_change_projects
    AFTER UPDATE OR DELETE ON app.projects
    FOR EACH ROW EXECUTE FUNCTION audit.log_row_change();

-- Триггер на app.contracts (критичная таблица)
CREATE TRIGGER audit_row_change_contracts
    AFTER UPDATE OR DELETE ON app.contracts
    FOR EACH ROW EXECUTE FUNCTION audit.log_row_change();

-- Триггер на app.actors (критичная таблица)
CREATE TRIGGER audit_row_change_actors
    AFTER UPDATE OR DELETE ON app.actors
    FOR EACH ROW EXECUTE FUNCTION audit.log_row_change();

-- Выдаём права на чтение логов аудита
GRANT SELECT ON audit.row_change_log TO auditor, security_admin,
app_writer, app_reader;
```

```
GRANT USAGE ON SCHEMA audit TO auditor, security_admin, app_writer,
app_reader;

-- Комментарии
COMMENT ON FUNCTION audit.log_row_change() IS
'Триггерная функция для логирования изменений строк в критичных
таблицах с маскированием чувствительных данных.';

COMMENT ON FUNCTION audit.mask_sensitive_data(JSONB, TEXT) IS
'Функция маскирования/хэширования чувствительных полей в данных
аудита.';

COMMENT ON TRIGGER audit_row_change_projects ON app.projects IS
'Триггер аудита изменений для таблицы проектов. Логирует UPDATE и
DELETE.';

COMMENT ON TRIGGER audit_row_change_contracts ON app.contracts IS
'Триггер аудита изменений для таблицы контрактов. Логирует UPDATE и
DELETE.';

COMMENT ON TRIGGER audit_row_change_actors ON app.actors IS
'Триггер аудита изменений для таблицы актёров. Логирует UPDATE и
DELETE.';

DO $$
BEGIN
    RAISE NOTICE 'LR4: Триггеры аудита изменений успешно созданы';
    RAISE NOTICE '  • Таблица: audit.row_change_log';
    RAISE NOTICE '  • Триггеры на: app.projects, app.contracts, app.actors';
    RAISE NOTICE '  • Маскирование: passport_data, tax_id, salary_amount,
terms, client_contacts, budget';
END $$;

RESET ROLE;
```

Приложение В

```
SET search_path TO app, ref, audit, stg, public;

DO $$
BEGIN
   RAISE NOTICE '━━━
```

```
        -- WITH CHECK OPTION должен заблокировать это
        INSERT INTO app.projects_public_view (
            studio_id,
            segment_id,
            title,
            original_language,
            status
        ) VALUES (
            1,
            2,  -- Чужой сегмент!
            'Test bypass WITH CHECK OPTION',
            'en',
            'draft'
        )
        RETURNING project_id INTO v_project_id;

        RAISE EXCEPTION '❌ ТЕСТ 1 ПРОВАЛЕН: удалось обойти WITH
CHECK OPTION (создан проект ID: %)', v_project_id;

    EXCEPTION
        WHEN others THEN
            IF SQLSTATE = '44000' OR SQLERRM LIKE '%WITH CHECK
OPTION%' OR SQLERRM LIKE '%row-level security%' THEN
                RAISE NOTICE '✅ ТЕСТ 1 ПРОЙДЕН: WITH CHECK OPTION
заблокировал вставку с чужим segment_id';
                RAISE NOTICE '   • Ошибка: %', SQLERRM;
            ELSE
                RAISE EXCEPTION '❌ ТЕСТ 1: Неожиданная ошибка: %',
SQLERRM;
            END IF;
    END;
END $$;

RESET ROLE;

--
==========================================================
====
-- ТЕСТ 2: Попытка получения деталей через SECURITY BARRIER VIEW
--
==========================================================
====
DO $$
BEGIN
```

```
    RAISE NOTICE '';
    RAISE NOTICE '━━━
```

```
    EXCEPTION
        WHEN OTHERS THEN
            -- Ошибка ожидаема, если SECURITY BARRIER работает правильно
            NULL;
    END;

    -- Проверяем, что VIEW возвращает только агрегаты
    IF v_row_count > 0 THEN
        RAISE NOTICE '✅ ТЕСТ 2 ПРОЙДЕН: SECURITY BARRIER VIEW
возвращает только агрегаты';
        RAISE NOTICE '   • Найдено агрегированных записей: %', v_row_count;
        RAISE NOTICE '   • Детали недоступны через VIEW (только агрегаты)';
    ELSE
        RAISE EXCEPTION '❌ ТЕСТ 2 ПРОВАЛЕН: VIEW не возвращает
данные';
    END IF;

    -- Дополнительная проверка: пытаемся получить детали через JOIN
    BEGIN
        SELECT COUNT(*) INTO v_row_count
        FROM app.projects_statistics_view psv
        JOIN app.projects p ON psv.segment_id = p.segment_id
        LIMIT 1;

        -- Если это работает, значит SECURITY BARRIER не блокирует JOIN
        -- Но это нормально, так как JOIN не раскрывает детали из VIEW

    EXCEPTION
        WHEN OTHERS THEN
            NULL;
    END;

END $$;

RESET ROLE;

--
==========================================================
====
-- ТЕСТ 3: Изменение строки - проверка появления записи в row_change_log
--
==========================================================
====
```

```
DO $$
BEGIN
   RAISE NOTICE '';
   RAISE NOTICE '━━━
```

```
    UPDATE app.projects
    SET status = v_new_status
    WHERE project_id = v_project_id;

    -- Подсчитываем записи в логе после обновления
    SELECT COUNT(*) INTO v_log_count_after
    FROM audit.row_change_log
    WHERE table_name = 'projects'
      AND record_id = v_project_id;

    -- Проверяем, что появилась новая запись
    IF v_log_count_after > v_log_count_before THEN
        -- Получаем последнюю запись
        SELECT * INTO v_log_record
        FROM audit.row_change_log
        WHERE table_name = 'projects'
          AND record_id = v_project_id
        ORDER BY change_time DESC
        LIMIT 1;

        RAISE NOTICE '✅ ТЕСТ 3 ПРОЙДЕН: Запись в audit.row_change_log
создана';
        RAISE NOTICE '   • Проект ID: %', v_project_id;
        RAISE NOTICE '   • Статус изменён: % → %', v_old_status, v_new_status;
        RAISE NOTICE '   • Записей в логе: было %, стало %',
v_log_count_before, v_log_count_after;
        RAISE NOTICE '   • Операция: %', v_log_record.operation_type;
        RAISE NOTICE '   • Изменено пользователем: %',
v_log_record.changed_by;

        -- Проверяем маскирование чувствительных данных
        IF v_log_record.old_values IS NOT NULL AND v_log_record.new_values
IS NOT NULL THEN
            RAISE NOTICE '   • Маскирование данных: применено (budget и
client_contacts замаскированы)';
        END IF;
    ELSE
        RAISE EXCEPTION '❌ ТЕСТ 3 ПРОВАЛЕН: запись в
audit.row_change_log не создана';
    END IF;

    -- Откатываем изменение
    UPDATE app.projects
    SET status = v_old_status
```

```
    WHERE project_id = v_project_id;

END $$;

RESET ROLE;

--
================================================================
====
-- ТЕСТ 4: Попытка удаления строки не из своего сегмента - ошибка RLS
--
================================================================
====
DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━';
    RAISE NOTICE 'ТЕСТ 4: Попытка удаления строки не из своего сегмента';
    RAISE NOTICE 'Роль: app_writer, Сегмент: 1';
    RAISE NOTICE 'Ожидается: ошибка RLS при попытке удалить проект из
сегмента 2';
END $$;

-- Получаем ID проекта из сегмента 2 ДО установки контекста сегмента
SET ROLE app_owner;
DO $$
DECLARE
    v_project_id_seg2 INTEGER;
BEGIN
    SELECT project_id INTO v_project_id_seg2
    FROM app.projects
    WHERE segment_id = 2
    ORDER BY project_id
    LIMIT 1;

    IF v_project_id_seg2 IS NULL THEN
        RAISE EXCEPTION '❌ ТЕСТ 4 ПРОВАЛЕН: не найден проект из
сегмента 2 для теста';
    END IF;

    -- Сохраняем в переменную сессии
    PERFORM set_config('test.project_id_seg2', v_project_id_seg2::TEXT, false);
```

```
END $$;
RESET ROLE;

-- Теперь пытаемся удалить проект из чужого сегмента
SET ROLE app_writer;
SELECT app.set_session_ctx(1, NULL);

DO $$
DECLARE
   v_project_id_seg2 INTEGER;
BEGIN
   -- Получаем сохранённый project_id
   v_project_id_seg2 := current_setting('test.project_id_seg2', true)::INTEGER;

   IF v_project_id_seg2 IS NULL THEN
      RAISE EXCEPTION '❌ ТЕСТ 4 ПРОВАЛЕН: не удалось получить
project_id из сегмента 2';
   END IF;

   BEGIN
      DELETE FROM app.projects
      WHERE project_id = v_project_id_seg2;

      RAISE EXCEPTION '❌ ТЕСТ 4 ПРОВАЛЕН: удалось удалить проект из
чужого сегмента (ID: %)', v_project_id_seg2;

   EXCEPTION
      WHEN others THEN
         IF SQLSTATE = '42501' OR SQLERRM LIKE '%row-level security%' OR
SQLERRM LIKE '%permission denied%' THEN
            RAISE NOTICE '✅ ТЕСТ 4 ПРОЙДЕН: RLS заблокировал удаление
проекта из чужого сегмента';
            RAISE NOTICE '   • Проект ID: % (сегмент 2)', v_project_id_seg2;
            RAISE NOTICE '   • Текущий сегмент: 1';
            RAISE NOTICE '   • Ошибка: %', SQLERRM;
         ELSE
            RAISE EXCEPTION '❌ ТЕСТ 4: Неожиданная ошибка: %',
SQLERRM;
         END IF;
   END;

END $$;

RESET ROLE;
```

Приложение Г

```
SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

--
================================================================
-- Создание таблицы для хранения результатов анализа
--
================================================================

GRANT USAGE ON SCHEMA audit TO app_writer, app_reader;

-- Удаляем старую таблицу, если она существует, чтобы пересоздать с
правильной структурой
DROP TABLE IF EXISTS audit.performance_analysis CASCADE;

CREATE TABLE audit.performance_analysis (
    analysis_id SERIAL PRIMARY KEY,
    analysis_time TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    query_name VARCHAR(200) NOT NULL,
    rls_enabled BOOLEAN NOT NULL,
    execution_time_ms NUMERIC(10, 3),
    planning_time_ms NUMERIC(10, 3),
    buffers_shared_hit BIGINT DEFAULT 0,
    buffers_shared_read BIGINT DEFAULT 0,
    buffers_local_hit BIGINT DEFAULT 0,
    buffers_local_read BIGINT DEFAULT 0,
    buffers_temp_read BIGINT DEFAULT 0,
    buffers_temp_written BIGINT DEFAULT 0,
    rows_examined BIGINT,
    rows_returned BIGINT,
    explain_plan TEXT,
    indexes_used TEXT[]
);

COMMENT ON TABLE audit.performance_analysis IS
'Результаты анализа производительности запросов до/после RLS';

GRANT SELECT, INSERT ON audit.performance_analysis TO app_writer,
app_reader, auditor, security_admin;
```

```
GRANT USAGE, SELECT ON SEQUENCE
audit.performance_analysis_analysis_id_seq TO app_writer, app_reader;
GRANT USAGE ON SCHEMA audit TO app_writer, app_reader;

--
==========================================================
====
-- Функция для парсинга EXPLAIN (ANALYZE, BUFFERS) и сохранения
результатов
--
==========================================================
====

CREATE OR REPLACE FUNCTION audit.save_explain_result(
    p_query_name TEXT,
    p_rls_enabled BOOLEAN,
    p_explain_text TEXT
)
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
DECLARE
    v_execution_time NUMERIC := NULL;
    v_planning_time NUMERIC := NULL;
    v_buffers_shared_hit BIGINT := 0;
    v_buffers_shared_read BIGINT := 0;
    v_buffers_local_hit BIGINT := 0;
    v_buffers_local_read BIGINT := 0;
    v_buffers_temp_read BIGINT := 0;
    v_buffers_temp_written BIGINT := 0;
    v_rows_examined BIGINT := NULL;
    v_rows_returned BIGINT := NULL;
    v_indexes_used TEXT[] := ARRAY[]::TEXT[];
    v_lines TEXT[];
    v_line TEXT;
    v_match TEXT;
BEGIN
    -- Парсим результаты из EXPLAIN (ANALYZE, BUFFERS)
    -- Ищем строки вида: "Planning Time: 0.123 ms"
    -- "Execution Time: 1.456 ms"
    -- "Buffers: shared hit=10 read=5"

    v_lines := string_to_array(p_explain_text, E'\n');
```

```
    FOREACH v_line IN ARRAY v_lines
    LOOP
        -- Парсим Planning Time (формат: "Planning Time: 0.319 ms")
        IF v_line ~* 'Planning Time:' THEN
            v_match := substring(v_line from 'Planning Time:\s*([0-9]+\.[0-9]+|[0-
9]+)\s*ms');
            IF v_match IS NOT NULL THEN
                v_planning_time := v_match::NUMERIC;
            END IF;
        END IF;

        -- Парсим Execution Time (формат: "Execution Time: 0.033 ms")
        IF v_line ~* 'Execution Time:' THEN
            v_match := substring(v_line from 'Execution Time:\s*([0-9]+\.[0-9]+|[0-
9]+)\s*ms');
            IF v_match IS NOT NULL THEN
                v_execution_time := v_match::NUMERIC;
            END IF;
        END IF;

        -- Парсим Buffers (может быть несколько строк)
        IF v_line ~* 'Buffers:' THEN
            -- shared hit
            v_match := substring(v_line from 'shared hit=(\d+)');
            IF v_match IS NOT NULL THEN
                v_buffers_shared_hit := v_buffers_shared_hit + v_match::BIGINT;
            END IF;
            -- shared read
            v_match := substring(v_line from 'shared read=(\d+)');
            IF v_match IS NOT NULL THEN
                v_buffers_shared_read := v_buffers_shared_read + v_match::BIGINT;
            END IF;
            -- local hit
            v_match := substring(v_line from 'local hit=(\d+)');
            IF v_match IS NOT NULL THEN
                v_buffers_local_hit := v_buffers_local_hit + v_match::BIGINT;
            END IF;
            -- local read
            v_match := substring(v_line from 'local read=(\d+)');
            IF v_match IS NOT NULL THEN
                v_buffers_local_read := v_buffers_local_read + v_match::BIGINT;
            END IF;
            -- temp read
            v_match := substring(v_line from 'temp read=(\d+)');
```

```
        IF v_match IS NOT NULL THEN
            v_buffers_temp_read := v_buffers_temp_read + v_match::BIGINT;
        END IF;
        -- temp written
        v_match := substring(v_line from 'temp written=(\d+)');
        IF v_match IS NOT NULL THEN
            v_buffers_temp_written := v_buffers_temp_written + v_match::BIGINT;
        END IF;
    END IF;

    -- Парсим индексы (ищем строки с Index Scan, Index Only Scan)
    IF v_line ~* 'Index (Only )?Scan' THEN
        v_match := substring(v_line from 'Index (Only )?Scan (?:using )?(\w+)');
        IF v_match IS NOT NULL THEN
            v_indexes_used := array_append(v_indexes_used, v_match);
        END IF;
    END IF;
END LOOP;

-- Пытаемся найти rows examined/returned из плана
-- Это упрощённая версия, в реальности нужно парсить весь план
v_rows_examined := NULL;
v_rows_returned := NULL;

INSERT INTO audit.performance_analysis (
    query_name,
    rls_enabled,
    execution_time_ms,
    planning_time_ms,
    buffers_shared_hit,
    buffers_shared_read,
    buffers_local_hit,
    buffers_local_read,
    buffers_temp_read,
    buffers_temp_written,
    rows_examined,
    rows_returned,
    explain_plan,
    indexes_used
) VALUES (
    p_query_name,
    p_rls_enabled,
    v_execution_time,
    v_planning_time,
    v_buffers_shared_hit,
```

```
        v_buffers_shared_read,
        v_buffers_local_hit,
        v_buffers_local_read,
        v_buffers_temp_read,
        v_buffers_temp_written,
        v_rows_examined,
        v_rows_returned,
        p_explain_text,
        v_indexes_used
    );
END;
$$;

--
==========================================================
====
-- Вспомогательная функция для выполнения EXPLAIN и сохранения
результатов
--
==========================================================
====

CREATE OR REPLACE FUNCTION audit.run_explain_test(
    p_query_name TEXT,
    p_query_sql TEXT,
    p_rls_enabled BOOLEAN
)
RETURNS TABLE (
    execution_time_ms NUMERIC,
    planning_time_ms NUMERIC,
    total_buffers_read BIGINT
)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
DECLARE
    v_explain_text TEXT := '';
    v_line TEXT;
    v_result RECORD;
    v_rec RECORD;
BEGIN
    -- Выполняем EXPLAIN (ANALYZE, BUFFERS) и собираем все строки
    FOR v_rec IN
```

```
    EXECUTE format('EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
%s', p_query_sql)
  LOOP
    v_explain_text := v_explain_text || v_rec."QUERY PLAN" || E'\n';
  END LOOP;

  -- Сохраняем результат
  PERFORM audit.save_explain_result(p_query_name, p_rls_enabled,
v_explain_text);

  -- Возвращаем метрики
  SELECT
    pa.execution_time_ms,
    pa.planning_time_ms,
    (pa.buffers_shared_read + pa.buffers_local_read)::BIGINT as
total_buffers_read
  INTO v_result
  FROM audit.performance_analysis pa
  WHERE pa.query_name = p_query_name
    AND pa.rls_enabled = p_rls_enabled
  ORDER BY pa.analysis_time DESC
  LIMIT 1;

  RETURN QUERY SELECT
    v_result.execution_time_ms,
    v_result.planning_time_ms,
    v_result.total_buffers_read;
END;
$$;

--
==========================================================
====
-- Создание индексов для оптимизации RLS политик
--
==========================================================
====

DO $$
BEGIN
  RAISE NOTICE '';
  RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━';
```

```
    RAISE NOTICE 'СОЗДАНИЕ ИНДЕКСОВ ДЛЯ УСЛОВИЙ ПОЛИТИК
RLS';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━';
END $$;

-- Индекс для projects (segment_id, status) - часто используется в политиках
CREATE INDEX IF NOT EXISTS idx_projects_segment_status
ON app.projects(segment_id, status);

-- Индекс для contracts (segment_id, actor_id)
CREATE INDEX IF NOT EXISTS idx_contracts_segment_actor
ON app.contracts(segment_id, actor_id);

-- Индекс для recording_sessions (segment_id, session_date)
CREATE INDEX IF NOT EXISTS idx_sessions_segment_date
ON app.recording_sessions(segment_id, session_date);

-- Индекс для actors (segment_id)
CREATE INDEX IF NOT EXISTS idx_actors_segment
ON app.actors(segment_id);

--
================================================================
====
-- ТЕСТ 1: Выборка проектов по segment_id и status
--
================================================================
====

DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━';
    RAISE NOTICE 'ТЕСТ 1: Выборка проектов по segment_id и status';
    RAISE NOTICE 'Запрос: SELECT * FROM app.projects WHERE segment_id
= 1 AND status = "active"';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━';
END $$;
```

```
-- Тест БЕЗ RLS
DO $$
DECLARE
    v_result RECORD;
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '▶️ Выполнение теста БЕЗ RLS...';

    -- Очищаем кэш планов для справедливого сравнения
    DISCARD PLANS;

    -- Временно отключаем RLS на таблице projects
    ALTER TABLE app.projects DISABLE ROW LEVEL SECURITY;

    -- Выполняем тест
    SELECT * INTO v_result
    FROM audit.run_explain_test(
        'TEST1: Projects by segment and status',
        'SELECT * FROM app.projects WHERE segment_id = 1 AND status =
"active"',
        FALSE
    );

    -- Включаем RLS обратно
    ALTER TABLE app.projects ENABLE ROW LEVEL SECURITY;

    IF v_result.execution_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время выполнения: % ms',
ROUND(v_result.execution_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время выполнения: не определено';
    END IF;
    IF v_result.planning_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время планирования: % ms',
ROUND(v_result.planning_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время планирования: не определено';
    END IF;
    RAISE NOTICE '   Буферов прочитано: %',
COALESCE(v_result.total_buffers_read, 0);
END $$;

-- Тест С RLS
DO $$
```

```
DECLARE
    v_result RECORD;
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '▶️ Выполнение теста С RLS...';

    -- Очищаем кэш планов для справедливого сравнения
    DISCARD PLANS;

    SET ROLE app_writer;
    PERFORM app.set_session_ctx(1, NULL);

    -- Выполняем тест
    SELECT * INTO v_result
    FROM audit.run_explain_test(
        'TEST1: Projects by segment and status',
        'SELECT * FROM app.projects WHERE segment_id = 1 AND status =
"active"',
        TRUE
    );

    RESET ROLE;

    IF v_result.execution_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время выполнения: % ms',
ROUND(v_result.execution_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время выполнения: не определено';
    END IF;
    IF v_result.planning_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время планирования: % ms',
ROUND(v_result.planning_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время планирования: не определено';
    END IF;
    RAISE NOTICE '   Буферов прочитано: %',
COALESCE(v_result.total_buffers_read, 0);
END $$;

--
==========================================================
====
-- ТЕСТ 2: Выборка контрактов с JOIN к актёрам
```

```
--
==========================================================
====

DO $$
BEGIN
   RAISE NOTICE '';
   RAISE NOTICE '━━━
```

```
    ALTER TABLE app.contracts ENABLE ROW LEVEL SECURITY;
    ALTER TABLE app.actors ENABLE ROW LEVEL SECURITY;

    IF v_result.execution_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время выполнения: % ms',
ROUND(v_result.execution_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время выполнения: не определено';
    END IF;
    IF v_result.planning_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время планирования: % ms',
ROUND(v_result.planning_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время планирования: не определено';
    END IF;
    RAISE NOTICE '   Буферов прочитано: %',
COALESCE(v_result.total_buffers_read, 0);
END $$;

-- Тест C RLS
DO $$
DECLARE
    v_result RECORD;
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '▶️ Выполнение теста C RLS...';

    -- Очищаем кэш планов для справедливого сравнения
    DISCARD PLANS;

    SET ROLE app_writer;
    PERFORM app.set_session_ctx(1, NULL);

    -- Выполняем тест
    SELECT * INTO v_result
    FROM audit.run_explain_test(
        'TEST2: Contracts with JOIN to actors',
        'SELECT c.*, a.first_name, a.last_name FROM app.contracts c JOIN
app.actors a ON c.actor_id = a.actor_id WHERE c.segment_id = 1',
        TRUE
    );

    RESET ROLE;

    IF v_result.execution_time_ms IS NOT NULL THEN
```

```
        RAISE NOTICE '   Время выполнения: % ms',
ROUND(v_result.execution_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время выполнения: не определено';
    END IF;
    IF v_result.planning_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время планирования: % ms',
ROUND(v_result.planning_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время планирования: не определено';
    END IF;
    RAISE NOTICE '   Буферов прочитано: %',
COALESCE(v_result.total_buffers_read, 0);
END $$;

--
================================================
====
-- ТЕСТ 3: Агрегирующий запрос по сессиям записи
--
================================================
====

DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '━━━
```

```
    RAISE NOTICE '▶️ Выполнение теста БЕЗ RLS...';

    -- Очищаем кэш планов для справедливого сравнения
    DISCARD PLANS;

    -- Временно отключаем RLS
    ALTER TABLE app.recording_sessions DISABLE ROW LEVEL SECURITY;

    -- Выполняем тест
    SELECT * INTO v_result
    FROM audit.run_explain_test(
        'TEST3: Aggregate query on recording_sessions',
        'SELECT segment_id, status, COUNT(*), SUM(duration_hours) FROM
app.recording_sessions GROUP BY segment_id, status',
        FALSE
    );

    -- Включаем RLS обратно
    ALTER TABLE app.recording_sessions ENABLE ROW LEVEL SECURITY;

    IF v_result.execution_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время выполнения: % ms',
ROUND(v_result.execution_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время выполнения: не определено';
    END IF;
    IF v_result.planning_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время планирования: % ms',
ROUND(v_result.planning_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время планирования: не определено';
    END IF;
    RAISE NOTICE '   Буферов прочитано: %',
COALESCE(v_result.total_buffers_read, 0);
END $$;

-- Тест C RLS
DO $$
DECLARE
    v_result RECORD;
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '▶️ Выполнение теста C RLS...';
```

```
    -- Очищаем кэш планов для справедливого сравнения
    DISCARD PLANS;

    SET ROLE app_writer;
    PERFORM app.set_session_ctx(1, NULL);

    -- Выполняем тест
    SELECT * INTO v_result
    FROM audit.run_explain_test(
        'TEST3: Aggregate query on recording_sessions',
        'SELECT segment_id, status, COUNT(*), SUM(duration_hours) FROM
app.recording_sessions GROUP BY segment_id, status',
        TRUE
    );

    RESET ROLE;

    IF v_result.execution_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время выполнения: % ms',
ROUND(v_result.execution_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время выполнения: не определено';
    END IF;
    IF v_result.planning_time_ms IS NOT NULL THEN
        RAISE NOTICE '   Время планирования: % ms',
ROUND(v_result.planning_time_ms::NUMERIC, 3);
    ELSE
        RAISE NOTICE '   Время планирования: не определено';
    END IF;
    RAISE NOTICE '   Буферов прочитано: %',
COALESCE(v_result.total_buffers_read, 0);
END $$;

--
================================================================
====
-- Сравнение производительности: вывод результатов
--
================================================================
====

DO $$
BEGIN
    RAISE NOTICE '';
```

```
    RAISE NOTICE '━━━
```

```
        -- Получаем результаты С RLS
        SELECT * INTO v_with_rls
        FROM audit.performance_analysis
        WHERE query_name = v_test_name AND rls_enabled = TRUE
        ORDER BY analysis_time DESC
        LIMIT 1;

        IF v_without_rls IS NOT NULL AND v_with_rls IS NOT NULL THEN
            v_overhead := ((v_with_rls.execution_time_ms -
v_without_rls.execution_time_ms) / NULLIF(v_without_rls.execution_time_ms,
0) * 100);

            RAISE NOTICE '';
            RAISE NOTICE '━━━
```

```
        END IF;
        IF v_with_rls.planning_time_ms IS NOT NULL THEN
            RAISE NOTICE '  • Время планирования: % мс',
ROUND(v_with_rls.planning_time_ms::NUMERIC, 3);
        ELSE
            RAISE NOTICE '  • Время планирования: не определено';
        END IF;
        RAISE NOTICE '  • Буферов прочитано: %',
COALESCE((v_with_rls.buffers_shared_read + v_with_rls.buffers_local_read),
0);
        RAISE NOTICE '';
        RAISE NOTICE 'ЭФФЕКТ RLS:';
        IF v_without_rls.execution_time_ms IS NOT NULL AND
v_with_rls.execution_time_ms IS NOT NULL THEN
            RAISE NOTICE '  • Прирост времени выполнения: %%%',
ROUND(v_overhead::NUMERIC, 2);
            RAISE NOTICE '  • Разница во времени: % мс',
ROUND((v_with_rls.execution_time_ms -
v_without_rls.execution_time_ms)::NUMERIC, 3);
        ELSE
            RAISE NOTICE '  • Прирост времени выполнения: не определено';
        END IF;
        RAISE NOTICE '  • Разница в буферах: %',
COALESCE(((v_with_rls.buffers_shared_read + v_with_rls.buffers_local_read) -
(v_without_rls.buffers_shared_read + v_without_rls.buffers_local_read)), 0);
      END IF;
   END LOOP;
END $$;

--
==========================================================
====
-- Функция для сравнения производительности
--
==========================================================
====

DROP FUNCTION IF EXISTS audit.compare_performance() CASCADE;

CREATE OR REPLACE FUNCTION audit.compare_performance()
RETURNS TABLE (
   query_name TEXT,
   rls_enabled BOOLEAN,
   execution_time_ms NUMERIC,
   planning_time_ms NUMERIC,
```

```
    total_buffers_read BIGINT,
    buffers_from_cache BIGINT
)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
BEGIN
    RETURN QUERY
    SELECT
        pa.query_name::TEXT,
        pa.rls_enabled,
        pa.execution_time_ms,
        pa.planning_time_ms,
        (pa.buffers_shared_read + pa.buffers_local_read)::BIGINT as
total_buffers_read,
        pa.buffers_shared_hit::BIGINT as buffers_from_cache
    FROM audit.performance_analysis pa
    WHERE pa.query_name LIKE 'TEST%'
    ORDER BY pa.query_name, pa.rls_enabled;
END;
$$;

GRANT EXECUTE ON FUNCTION audit.save_explain_result(TEXT,
BOOLEAN, TEXT) TO app_reader, app_writer, auditor, security_admin;
GRANT EXECUTE ON FUNCTION audit.run_explain_test(TEXT, TEXT,
BOOLEAN) TO app_reader, app_writer, auditor, security_admin;
GRANT EXECUTE ON FUNCTION audit.compare_performance() TO
app_reader, app_writer, auditor, security_admin;

RESET ROLE;
```

Приложение Д

```
SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

--
=============================================================
====
-- Создание архивной таблицы
--
=============================================================
====

CREATE TABLE IF NOT EXISTS audit.row_change_log_archive (
    log_id BIGINT PRIMARY KEY,
    change_time TIMESTAMPTZ NOT NULL,
    changed_by VARCHAR(100) NOT NULL,
    table_schema VARCHAR(100) NOT NULL,
    table_name VARCHAR(100) NOT NULL,
    operation_type VARCHAR(10) NOT NULL,
    record_id INTEGER,
    old_values JSONB,
    new_values JSONB,
    changed_fields TEXT[],
    client_ip INET,
    session_info JSONB,
    archived_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- Индексы для архивной таблицы
CREATE INDEX IF NOT EXISTS idx_row_change_log_archive_time
ON audit.row_change_log_archive(change_time);
CREATE INDEX IF NOT EXISTS idx_row_change_log_archive_table
ON audit.row_change_log_archive(table_schema, table_name);
CREATE INDEX IF NOT EXISTS idx_row_change_log_archive_user
ON audit.row_change_log_archive(changed_by);
CREATE INDEX IF NOT EXISTS idx_row_change_log_archive_archived
ON audit.row_change_log_archive(archived_at);

COMMENT ON TABLE audit.row_change_log_archive IS
'Архивная таблица для старых записей из audit.row_change_log. Записи
старше N дней переносятся сюда.';
```

```
--
============================================================
====
-- Функция резервного копирования audit-логов
--
============================================================
====

CREATE OR REPLACE FUNCTION audit.backup_audit_logs(days_interval
INT)
RETURNS TABLE (
    records_moved BIGINT,
    oldest_record_date TIMESTAMPTZ,
    newest_record_date TIMESTAMPTZ,
    backup_time TIMESTAMPTZ
)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
DECLARE
    v_cutoff_date TIMESTAMPTZ;
    v_records_moved BIGINT;
    v_oldest_date TIMESTAMPTZ;
    v_newest_date TIMESTAMPTZ;
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_duration INTERVAL;
BEGIN
    v_start_time := clock_timestamp();

    -- Вычисляем дату отсечки
    v_cutoff_date := NOW() - (days_interval || ' days')::INTERVAL;

    RAISE NOTICE '';
    RAISE NOTICE '━━━
```

```
    RAISE NOTICE '  • Интервал архивирования: % дней', days_interval;
    RAISE NOTICE '  • Дата отсечки: %', v_cutoff_date;
    RAISE NOTICE '';

    -- Получаем информацию о записях, которые будут перемещены
    SELECT
        COUNT(*),
        MIN(change_time),
        MAX(change_time)
    INTO
        v_records_moved,
        v_oldest_date,
        v_newest_date
    FROM audit.row_change_log
    WHERE change_time < v_cutoff_date;

    RAISE NOTICE 'Анализ записей для архивирования:';
    RAISE NOTICE '  • Записей для перемещения: %', v_records_moved;

    IF v_records_moved = 0 THEN
        RAISE NOTICE '';
        RAISE NOTICE '⚠️  Нет записей для архивирования (все записи новее % дней)', days_interval;
        RAISE NOTICE '';

        RETURN QUERY SELECT
            0::BIGINT,
            NULL::TIMESTAMPTZ,
            NULL::TIMESTAMPTZ,
            clock_timestamp();
        RETURN;
    END IF;

    IF v_oldest_date IS NOT NULL THEN
        RAISE NOTICE '  • Старейшая запись: %', v_oldest_date;
    END IF;
    IF v_newest_date IS NOT NULL THEN
        RAISE NOTICE '  • Новейшая запись: %', v_newest_date;
    END IF;
    RAISE NOTICE '';

    -- Перемещаем записи в архив
    RAISE NOTICE '▶️ Перемещение записей в архивную таблицу...';
```

```
INSERT INTO audit.row_change_log_archive (
    log_id,
    change_time,
    changed_by,
    table_schema,
    table_name,
    operation_type,
    record_id,
    old_values,
    new_values,
    changed_fields,
    client_ip,
    session_info
)
SELECT
    log_id,
    change_time,
    changed_by,
    table_schema,
    table_name,
    operation_type,
    record_id,
    old_values,
    new_values,
    changed_fields,
    client_ip,
    session_info
FROM audit.row_change_log
WHERE change_time < v_cutoff_date
ON CONFLICT (log_id) DO NOTHING;

RAISE NOTICE ' ✅ Записи перемещены в архив';

-- Удаляем перемещённые записи из основной таблицы
RAISE NOTICE '▶️ Удаление записей из основной таблицы...';

DELETE FROM audit.row_change_log
WHERE change_time < v_cutoff_date;

RAISE NOTICE ' ✅ Записи удалены из основной таблицы';

v_end_time := clock_timestamp();
v_duration := v_end_time - v_start_time;
```

```
    -- Возвращаем результаты
    RETURN QUERY SELECT
        v_records_moved,
        v_oldest_date,
        v_newest_date,
        v_end_time;

    -- Итоговая информация
    RAISE NOTICE '';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━';
    RAISE NOTICE 'РЕЗЕРВНОЕ КОПИРОВАНИЕ ЗАВЕРШЕНО
УСПЕШНО';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━';
    RAISE NOTICE 'Результаты:';
    RAISE NOTICE '  • Перемещено записей: %', v_records_moved;
    RAISE NOTICE '  • Время выполнения: %', v_duration;
    RAISE NOTICE '  • Завершено в: %', v_end_time;
    RAISE NOTICE '';

EXCEPTION
    WHEN OTHERS THEN
        RAISE EXCEPTION 'Ошибка при резервном копировании: %',
SQLERRM;
END;
$$;

--
==========================================================
====
-- Функция для просмотра статистики архива
--
==========================================================
====

CREATE OR REPLACE FUNCTION audit.get_archive_statistics()
RETURNS TABLE (
    total_archived BIGINT,
    oldest_record TIMESTAMPTZ,
    newest_record TIMESTAMPTZ,
    records_by_table JSONB,
```

```
    records_by_operation JSONB
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_records_by_table JSONB;
    v_records_by_operation JSONB;
BEGIN
    -- Статистика по таблицам
    SELECT jsonb_object_agg(table_name, count) INTO v_records_by_table
    FROM (
        SELECT
            table_name,
            COUNT(*)::BIGINT as count
        FROM audit.row_change_log_archive
        GROUP BY table_name
        ORDER BY count DESC
    ) stats;

    -- Статистика по операциям
    SELECT jsonb_object_agg(operation_type, count) INTO
v_records_by_operation
    FROM (
        SELECT
            operation_type,
            COUNT(*)::BIGINT as count
        FROM audit.row_change_log_archive
        GROUP BY operation_type
        ORDER BY count DESC
    ) stats;

    RETURN QUERY
    SELECT
        COUNT(*)::BIGINT as total_archived,
        MIN(change_time) as oldest_record,
        MAX(change_time) as newest_record,
        COALESCE(v_records_by_table, '{}'::JSONB) as records_by_table,
        COALESCE(v_records_by_operation, '{}'::JSONB) as records_by_operation
    FROM audit.row_change_log_archive;
END;
$$;

-- Выдаём права
GRANT EXECUTE ON FUNCTION audit.backup_audit_logs(INT) TO
dml_admin, security_admin;
```

```
GRANT EXECUTE ON FUNCTION audit.get_archive_statistics() TO auditor,
security_admin;
GRANT SELECT ON audit.row_change_log_archive TO auditor, security_admin;

COMMENT ON FUNCTION audit.backup_audit_logs(INT) IS
'Переносит записи старше N дней из audit.row_change_log в
audit.row_change_log_archive и удаляет их из основной таблицы.';

COMMENT ON FUNCTION audit.get_archive_statistics() IS
'Возвращает статистику по архивным записям: общее количество, диапазон
дат, распределение по таблицам и операциям.';

--
==========================================================
====
-- Демонстрация работы функции
--
==========================================================
====

DO $$
BEGIN
   RAISE NOTICE '';
   RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━';
   RAISE NOTICE 'ДЕМОНСТРАЦИЯ: Резервное копирование записей
старше 30 дней';
   RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━';
END $$;

-- Сначала проверяем текущее состояние
DO $$
DECLARE
   v_total_records BIGINT;
   v_old_records BIGINT;
   v_archive_records BIGINT;
   v_oldest_record TIMESTAMPTZ;
   v_newest_record TIMESTAMPTZ;
BEGIN
   SELECT COUNT(*) INTO v_total_records FROM audit.row_change_log;
   SELECT COUNT(*) INTO v_old_records
```

```
    FROM audit.row_change_log
    WHERE change_time < NOW() - INTERVAL '30 days';
    SELECT COUNT(*) INTO v_archive_records FROM
audit.row_change_log_archive;

    SELECT MIN(change_time), MAX(change_time)
    INTO v_oldest_record, v_newest_record
    FROM audit.row_change_log;

    RAISE NOTICE '';
    RAISE NOTICE 'ТЕКУЩЕЕ СОСТОЯНИЕ:';
    RAISE NOTICE '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
━━━━━━━━━━';
    RAISE NOTICE 'Основная таблица (audit.row_change_log):';
    RAISE NOTICE '  • Всего записей: %', v_total_records;
    IF v_oldest_record IS NOT NULL THEN
        RAISE NOTICE '  • Старейшая запись: %', v_oldest_record;
    END IF;
    IF v_newest_record IS NOT NULL THEN
        RAISE NOTICE '  • Новейшая запись: %', v_newest_record;
    END IF;
    RAISE NOTICE '  • Записей старше 30 дней: %', v_old_records;
    RAISE NOTICE '';
    RAISE NOTICE 'Архивная таблица (audit.row_change_log_archive):';
    RAISE NOTICE '  • Всего записей в архиве: %', v_archive_records;
    RAISE NOTICE '';
END $$;

-- Выполняем резервное копирование
DO $$
DECLARE
    v_result RECORD;
    v_stats RECORD;
BEGIN
    -- Вызываем функцию резервного копирования
    SELECT * INTO v_result
    FROM audit.backup_audit_logs(30);

    -- Получаем статистику архива
    SELECT * INTO v_stats
    FROM audit.get_archive_statistics();

    RAISE NOTICE '';
```

```
    RAISE NOTICE '━━━
```

```
    RAISE NOTICE '━━━
```